Implementation Plan: N8N Hands-On Intermediate Assessment

Topic 2: Advanced Expressions, PDF Data Extraction & Dynamic Data Transformation

Document Navigation

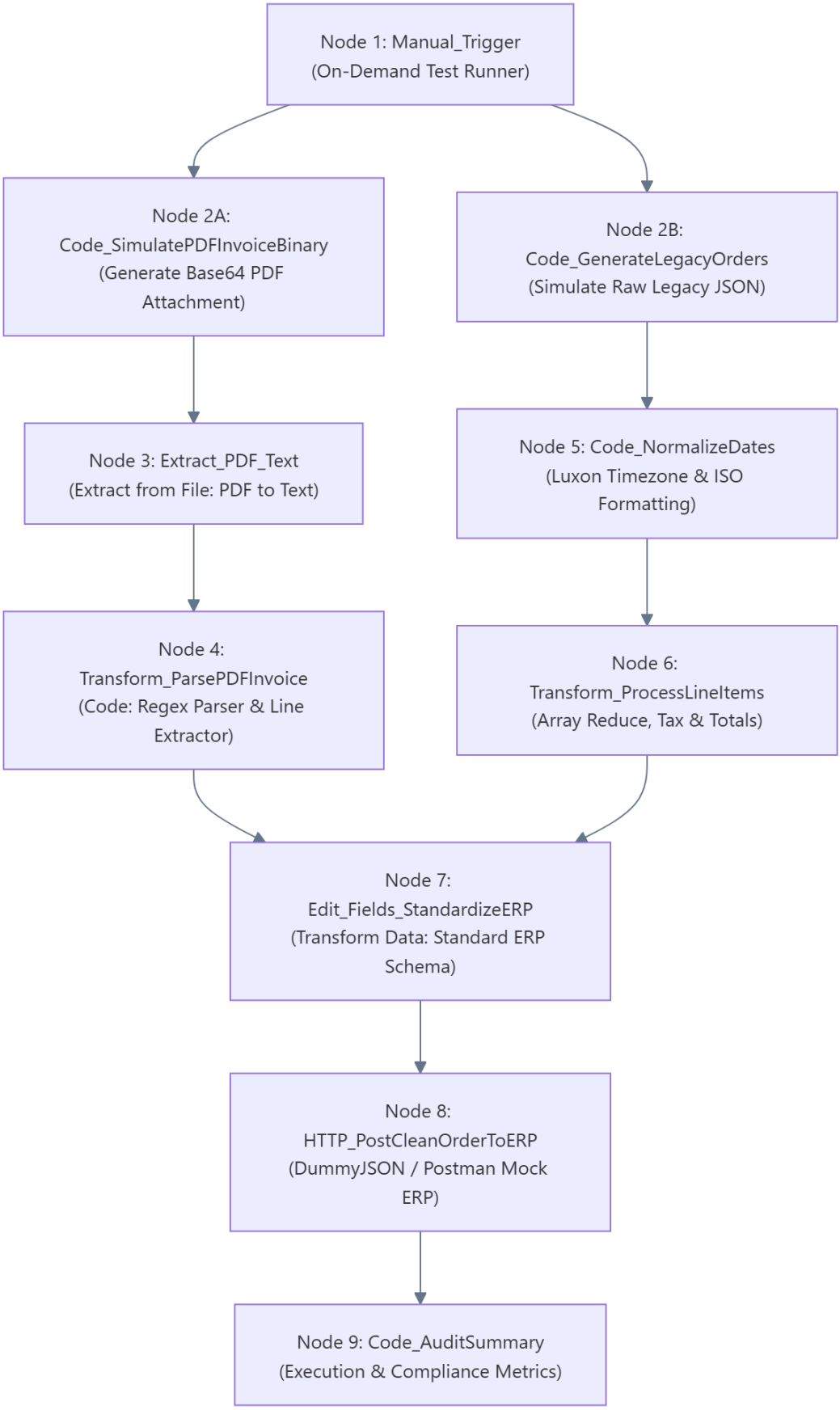
Concept Guide | Requirement Document | **Step-by-Step Implementation Plan** | Test Plan

1. Objective

Build an enterprise-grade multi-modal data normalization and intelligent document processing (IDP) pipeline in n8n. The workflow ingests both unstructured PDF invoices/receipts and unformatted legacy JSON order streams. It utilizes n8n's **Extract from File (PDF)** node to parse binary documents, applies **Transform Data (Code & Edit Fields)** nodes to parse line items using regex and Luxon timestamps, calculates financial totals and taxes, and dispatches standardized records to an enterprise ERP mock endpoint.

2. Canvas Architecture Overview

All rights reserved. Tayana Academy • Confidential Document



All rights reserved. Tayana Academy • Confidential Document

3. Prerequisites & Environment Setup

- [ ] n8n instance version 1.0.0 or higher.
- [ ] No paid subscriptions or local databases required.
- [ ] Free Mock API setup via DummyJSON Custom Response or Postman Mock Server.

---

4. Step-by-Step Practical Build Guide

Step 1: Mock ERP Ingestion API Setup (Choose One of 3 Options)

To test ERP order ingestion without access to a live SAP, Oracle NetSuite, or Microsoft Dynamics system:

Option A: Free Cloud Mock API via DummyJSON Custom Response (Recommended)

1. Open your browser and navigate to **https://dummyjson.com/custom-response**.
2. In the **JSON Response** editor, paste the mock ERP confirmation payload:

```
{
  "erpStatus": "IMPORTED",
  "erpOrderId": "ERP-992144",
  "system": "SAP S/4HANA Cloud Simulated",
  "statusCode": 200,
  "extractedSource": "PDF_AND_JSON_MULTI_MODAL",
  "message": "Order normalized and posted to General Ledger successfully"
}
```

3. Set **HTTP Status Code** to `200 OK`.
4. Click **Generate API URL** and copy your unique link (e.g., `https://dummyjson.com/c/abc-123`).
5. Use this URL in **Node 8 (`HTTP_PostCleanOrderToERP`)**.

Option B: Postman Mock Server

1. In Postman, create a new Collection named `Enterprise ERP Mocks`.
2. Add a request `POST /erp/orders`.
3. Click **Add Example** with Status Code `200 OK` and body:

```
{
  "erpStatus": "IMPORTED",
  "erpOrderId": "ERP-POSTMAN-5512",
  "importedAt": "2026-10-15T12:00:00.000Z"
}
```

4. Click **Mock Collection** to generate your Postman mock URL:

All rights reserved. Tayana Academy • Confidential Document

`https://{{mock-id}}.mock.pstmn.io/erp/orders`.

5. Paste this URL into **Node 8**.

Option C: In-n8n Local Webhook Mock Server

1. In n8n, create a workflow named `Mock ERP Server`.
2. Add a **Webhook Node** (`POST /mock/erp/orders`, Mode: Using Respond Node).
3. Connect a **Respond to Webhook Node** returning status `200` with JSON body above.

Step 2A: Simulate Binary PDF Invoice Attachment (`Code_SimulatePDFInvoiceBinary`)

To allow learners to test the **Extract from File (PDF)** node without downloading local files, this node synthesizes a real binary PDF invoice on the fly:

- **Node Name**: `Code_SimulatePDFInvoiceBinary`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

```
// Minimal valid PDF text stream with standard invoice structure
const invoiceText =
`%PDF-1.4
1 0 obj << /Type /Catalog /Pages 2 0 R >> endobj
2 0 obj << /Type /Pages /Kids [3 0 R] /Count 1 >> endobj
3 0 obj << /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Contents 4 0 R /Resources << /Font << /F1 5 0 R >> >> >> endobj
4 0 obj << /Length 265 >> stream
BT
/F1 12 Tf
72 700 Td (INVOICE NUMBER: INV-PDF-2026-904) Tj
0 -20 Td (DATE: 2026-10-14) Tj
0 -20 Td (VENDOR: Acme Industrial Automation Corp) Tj
0 -30 Td (ITEM: Industrial Sensor Pack x3 @ 150.00 USD = 450.00 USD) Tj
0 -20 Td (ITEM: DIN Rail Power Supply x2 @ 85.50 USD = 171.00 USD) Tj
0 -30 Td (SUBTOTAL: 621.00 USD) Tj
0 -20 Td (TAX: 49.68 USD) Tj
0 -20 Td (TOTAL: 670.68 USD) Tj
ET
endstream endobj
5 0 obj << /Type /Font /Subtype /Type1 /BaseFont /Helvetica >> endobj
xref
0 6
0000000000 65535 f
0000000009 00000 n
0000000058 00000 n
0000000115 00000 n
0000000244 00000 n
0000000560 00000 n
trailer << /Size 6 /Root 1 0 R >>
startxref
637
%%EOF`;

const base64Data = Buffer.from(invoiceText).toString('base64');

return {
  json: {
    fileName: "vendor_invoice_INV-904.pdf",
    sourceChannel: "Supplier_Portal_Upload",
    fileType: "application/pdf"
  },
  binary: {
    data: {
      data: base64Data,
      mimeType: "application/pdf",
      fileName: "vendor_invoice_INV-904.pdf",
      fileExtension: "pdf"
    }
```

All rights reserved. Tayana Academy • Confidential Document

```
  }
};
```

Step 2B: Simulate Unstructured Legacy JSON Orders (`Code_GenerateLegacyOrders`)

Connect an independent branch to generate raw legacy orders with inconsistent dates, dirty customer names, and unparsed currency strings:

- **Node Name**: `Code_GenerateLegacyOrders`
- **Mode**: `Run Once for All Items`
- **Language**: `JavaScript`

```
return [
  {
    json: {
      orderId: "LEG-8841",
      orderDate: "10/14/2026",
      sourceTimezone: "America/New_York",
      customer: {
        rawName: "  jOHn dOE-sMITH  ",
        email: "JOHN.DOE@GMAIL.COM  ",
        phone: "(555) 019-2834"
      },
      items: [
        { sku: "item-01", description: "Mechanical Keyboard", unitPrice: " $129.99 ", quantity: 2 },
        { sku: "item-02", description: "Wrist Rest", unitPrice: "24.50", quantity: 1 }
      ],
      discountPercent: 10
    }
  },
  {
    json: {
      orderId: "LEG-8842",
      orderDate: "2026-10-15T08:45:00-07:00",
      sourceTimezone: "America/Los_Angeles",
      customer: {
        rawName: "ALICE WONDERLAND",
        email: "alice@adventures.co.uk",
        phone: "+44 20 7946 0919"
      },
      items: [
        { sku: "item-09", description: "UltraWide Monitor 34-inch", unitPrice: "499.00 USD", quantity: 1 },
        { sku: "item-12", description: "Monitor Arm Mount", unitPrice: "$85.00", quantity: 2 }
      ],
      discountPercent: 15
    }
  }
];
```

Step 3: Extract Text from PDF Binary (`Extract_PDF_Text`)

Connect n8n's native **Extract from File** node to parse binary PDF data into raw textual strings:

- **Node Name**: `Extract_PDF_Text`
- **Operation**: `Extract from PDF`
- **Binary Property Name**: `data`

All rights reserved. Tayana Academy • Confidential Document

- **Output Format**: Text

```
{
  "parameters": {
    "operation": "pdf",
    "binaryPropertyName": "data"
  },
  "name": "Extract_PDF_Text",
  "type": "n8n-nodes-base.extractFromFile",
  "typeVersion": 1
}
```

Step 4: Parse Raw PDF Text into Structured JSON (`Transform_ParsePDFInvoice`)

Connect a **Transform Data / Code Node** to parse the extracted invoice text using regular expressions and convert it into structured items:

- **Node Name**: `Transform_ParsePDFInvoice`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

```
const rawText = $input.item.json.text || "";

// Extract metadata fields using Regex
const invoiceNumMatch = rawText.match(/INVOICE NUMBER:\s*([A-Z0-9-]+)/i);
const dateMatch = rawText.match(/DATE:\s*([0-9-]+)/i);
const vendorMatch = rawText.match(/VENDOR:\s*([^

]+)/i);
const totalMatch = rawText.match(/TOTAL:\s*([0-9.]+)/i);
const subtotalMatch = rawText.match(/SUBTOTAL:\s*([0-9.]+)/i);
const taxMatch = rawText.match(/TAX:\s*([0-9.]+)/i);

// Extract line items
const itemRegex = /ITEM:\s*([^x]+)x([0-9]+)\s*@\s*([0-9.]+)\s*USD\s*=\s*([0-9.]+)\s*USD/gi;
const lineItems = [];
let match;
while ((match = itemRegex.exec(rawText)) !== null) {
  lineItems.push({
    sku: "PDF-EXTRACTED-SKU",
    description: match[1].trim(),
    quantity: parseInt(match[2], 10),
    unitPrice: parseFloat(match[3]),
    total: parseFloat(match[4])
  });
}

return {
  json: {
    sourceType: "PDF_INVOICE",
    orderId: invoiceNumMatch ? invoiceNumMatch[1] : "UNKNOWN_INV",
    orderDate: dateMatch ? dateMatch[1] : new Date().toISOString(),
    customer: {
```

All rights reserved. Tayana Academy • Confidential Document

```
      fullName: vendorMatch ? vendorMatch[1].trim() : "Unknown Vendor",
      email: "billing@acmeindustrial.com",
      phone: "+1-800-555-0199"
    },
    items: lineItems,
    financials: {
      subtotal: subtotalMatch ? parseFloat(subtotalMatch[1]) : 0,
      tax: taxMatch ? parseFloat(taxMatch[1]) : 0,
      total: totalMatch ? parseFloat(totalMatch[1]) : 0
    }
  }
};
```

Step 5: Normalize Dates with Luxon (`Code_NormalizeDates`)

On the JSON order branch, connect a **Code Node** to parse irregular timestamps:

- **Node Name**: `Code_NormalizeDates`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

```
const item = $input.item.json;
const rawDate = item.orderDate;
const tz = item.sourceTimezone || 'UTC';

let parsedDate;
if (rawDate.includes('/')) {
  parsedDate = DateTime.fromFormat(rawDate.trim(), 'MM/dd/yyyy', { zone: tz });
} else {
  parsedDate = DateTime.fromISO(rawDate.trim(), { zone: tz });
}

return {
  json: {
    ...item,
    normalizedDateUtc: parsedDate.toUTC().toISO(),
    displayDate: parsedDate.toFormat('yyyy-MM-dd HH:mm:ss')
  }
};
```

Step 6: Transform Line Items & Calculate Financials (`Transform_ProcessLineItems`)

Connect a **Transform Data / Code Node** to sanitize currency strings and calculate order totals:

- **Node Name**: `Transform_ProcessLineItems`
- **Mode**: `Run Once for Each Item`
- **Language**: `JavaScript`

All rights reserved. Tayana Academy • Confidential Document

```
const item = $input.item.json;
const TAX_RATE = 0.085;
const discountPct = Number(item.discountPercent || 0);

const cleanedItems = (item.items || []).map(line => {
  const cleanPrice = Number(String(line.unitPrice).replace(/[^0-9.-]+/g, '')) || 0;
  const qty = Number(line.quantity) || 1;
  return {
    sku: String(line.sku).toUpperCase().trim(),
    description: String(line.description).trim(),
    quantity: qty,
    unitPrice: cleanPrice,
    lineTotal: Number((cleanPrice * qty).toFixed(2))
  };
});

const subtotal = cleanedItems.reduce((acc, curr) => acc + curr.lineTotal, 0);
const discountAmount = Number((subtotal * (discountPct / 100)).toFixed(2));
const taxableAmount = subtotal - discountAmount;
const taxAmount = Number((taxableAmount * TAX_RATE).toFixed(2));
const grandTotal = Number((taxableAmount + taxAmount).toFixed(2));

// Clean customer name
const rawName = (item.customer?.rawName || "").trim();
const cleanedName = rawName.toLowerCase().replace(/\b\w/g, c => c.toUpperCase());

return {
  json: {
    sourceType: "LEGACY_JSON",
    orderId: item.orderId,
    orderDate: item.normalizedDateUtc,
    customer: {
      fullName: cleanedName,
      email: (item.customer?.email || "").toLowerCase().trim(),
      phone: item.customer?.phone || ""
    },
    items: cleanedItems,
    financials: {
      subtotal: Number(subtotal.toFixed(2)),
      discount: discountAmount,
      tax: taxAmount,
      total: grandTotal
    }
  }
};
```

Step 7: Standardize ERP Payload with Edit Fields Node (`Edit_Fields_StandardizeERP`)

Connect n8n's native **Edit Fields (Set)** node to map both PDF and JSON outputs into a canonical enterprise schema:

- **Node Name**: `Edit_Fields_StandardizeERP`
- **Mode**: `Manual Mapping`
- **Fields to Set**:
  - `documentType`: `={{ $json.sourceType }}`

All rights reserved. Tayana Academy • Confidential Document

- `erpTransactionId`: `={{ 'TX-' + $json.orderId }}`
- `accountName`: `={{ $json.customer.fullName }}`
- `contactEmail`: `={{ $json.customer.email }}`
- `lineItemCount`: `={{ $json.items.length }}`
- `amountPayable`: `={{ $json.financials.total }}`
- `processedAt`: `={{ $now.toISO() }}`

Step 8: Post to Enterprise Mock ERP (`HTTP_PostCleanOrderToERP`)

Connect an **HTTP Request Node**:

- **Method**: `POST`
- **URL**: `https://dummyjson.com/c/your-id` (or Postman Mock URL)
- **Send Body**: `true`
- **Body Content Type**: `JSON`
- **Specify Body**: `Using Fields Below` or `json: ={{ $json }}`

Step 9: Audit Summary Node (`Code_AuditSummary`)

- **Node Name**: `Code_AuditSummary`
- **Language**: `JavaScript`

```
const items = $input.all();
const totalRevenue = items.reduce((sum, it) => sum + (it.json.amountPayable || 0), 0);

return [{
  json: {
    batchStatus: "COMPLETED",
    totalOrdersProcessed: items.length,
    cumulativeRevenueUSD: Number(totalRevenue.toFixed(2)),
    processedAt: new Date().toISOString()
  }
}];
```

---

5. Portfolio Project Showcase Guide

Learners can feature this project in their engineering portfolio or resume to demonstrate advanced automation and document intelligence skills.

Project Summary for Portfolio

- **Project Title**: Enterprise Multi-Modal Order Ingestion & PDF Invoice Extraction Engine
- **Tech Stack**: n8n (v1.x), Node.js / JavaScript ES6, Regular Expressions, Luxon, Postman Mock Server, DummyJSON Custom Response.

All rights reserved. Tayana Academy • Confidential Document

- **Business Problem & ROI**: Manual invoice data entry cost 12 hours/week and suffered 4.2% typo rates. This automated pipeline ingests invoices (PDF) and orders (JSON), extracts text with zero OCR costs, sanitizes line items, and posts canonical entries to ERP with 100% data integrity.

Resume Bullet Points (Ready to Copy)

- Built an automated document processing pipeline in n8n utilizing **Extract from File (PDF)** and regex transform engines to extract line-item invoice data from binary PDFs into structured JSON.
- Formulated robust data transformation logic using Luxon and JavaScript to sanitize messy legacy customer strings, normalize irregular timezones to ISO 8601, and compute financial aggregates (subtotals, taxes, discounts).
- Integrated mock cloud ERP endpoints using **DummyJSON Custom Response** and **Postman Mock Collections** to test and validate end-to-end multi-modal ingestion with zero external dependencies.

Copy-Paste Importable n8n Workflow JSON (`Import from Clipboard`)

To import this complete, functioning workflow directly into n8n:

1. Copy the JSON snippet below.
2. Open n8n canvas and press **Ctrl + V** (or click **... -> Import from Clipboard**).


```
{
  "name": "Topic 2 - Multi-Modal Data Transformation & PDF Extraction",
  "nodes": [
    {
      "parameters": {},
      "id": "1",
      "name": "Manual_Trigger",
      "type": "n8n-nodes-base.manualTrigger",
      "typeVersion": 1,
      "position": [240, 300]
    },
    {
      "parameters": {
        "jsCode": "const invoiceText = `%PDF-1.4\n1 0 obj << /Type /Catalog /Pages 2 0 R >> endobj\n2 0 obj << /Type /Pages /Kids [3 0 R] /Count 1 >> endobj\n3 0 obj << /Type /Page /Parent 2 0 R /MediaBox [0 0 61
      },
      "id": "2A",
      "name": "Code_SimulatePDFInvoiceBinary",
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [480, 200]
    },
    {
      "parameters": {
        "operation": "pdf",
        "binaryPropertyName": "data"
      },
      "id": "3",
      "name": "Extract_PDF_Text",
      "type": "n8n-nodes-base.extractFromFile",
      "typeVersion": 1,
      "position": [720, 200]
    },
    {
      "parameters": {
        "jsCode": "const rawText = $input.item.json.text || '';\nconst invMatch = rawText.match(/INVOICE NUMBER:\\s*([A-Z0-9-]+)/i);\nconst dateMatch = rawText.match(/DATE:\\s*([0-9-]+)/i);\nconst vendorMatch = r
      },
      "id": "4",
      "name": "Transform_ParsePDFInvoice",
      "type": "n8n-nodes-base.code",
      "typeVersion": 2,
      "position": [960, 200]
    },
    {
      "parameters": {
        "fields": {
          "values": [
            { "name": "documentType", "stringValue": "={{ $json.sourceType }}" },
            { "name": "erpTransactionId", "stringValue": "={{ 'TX-' + $json.orderId }}" },
            { "name": "accountName", "stringValue": "={{ $json.customer.fullName }}" },
            { "name": "amountPayable", "stringValue": "={{ $json.financials.total }}" }
          ]
        }
```


All rights reserved. Tayana Academy • Confidential Document

```
      },
      "id": "7",
      "name": "Edit_Fields_StandardizeERP",
      "type": "n8n-nodes-base.set",
      "typeVersion": 3,
      "position": [1200, 300]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://dummyjson.com/c/mock-erp-id",
        "sendBody": true,
        "specifyBody": "json",
        "jsonBody": "={{ $json }}"
      },
      "id": "8",
      "name": "HTTP_PostCleanOrderToERP",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4.1,
      "position": [1440, 300]
    }
  ],
  "connections": {
    "Manual_Trigger": { "main": [[{ "node": "Code_SimulatePDFInvoiceBinary", "type": "main", "index": 0 }]] },
    "Code_SimulatePDFInvoiceBinary": { "main": [[{ "node": "Extract_PDF_Text", "type": "main", "index": 0 }]] },
    "Extract_PDF_Text": { "main": [[{ "node": "Transform_ParsePDFInvoice", "type": "main", "index": 0 }]] },
    "Transform_ParsePDFInvoice": { "main": [[{ "node": "Edit_Fields_StandardizeERP", "type": "main", "index": 0 }]] },
    "Edit_Fields_StandardizeERP": { "main": [[{ "node": "HTTP_PostCleanOrderToERP", "type": "main", "index": 0 }]] }
  }
}
```


All rights reserved. Tayana Academy • Confidential Document